

Web Application Security Assessment

Northstar Ticketing Application

Assessment type: Authorized web application security assessment

Prepared by: Arky Craft
Date: September 2026

This report documents the methodology, findings, and recommendations from a targeted security assessment of the Northstar web application in a controlled lab environment.

Contents

Contents	2
1. Executive Summary.....	3
2. Scope and Rules	4
3. Environment and Methodology	5
4. Findings Summary.....	7
5. Detailed Findings	8
F-01: Insecure Direct Object Reference (IDOR) in Ticket Detail Endpoint.....	8
F-02: Incomplete Session-Cookie Security Attributes and HTTP Transport	10
T-01: SQL Injection Testing of Login Endpoint — No Exploitation Observed	11
T-02: Vertical Privilege Escalation — No Exploitation Observed	12
T-03: Reflected XSS Testing — No Exploitation Observed	13
T-04: CSRF Testing Limitations.....	14
T-05: Password-Storage Review Limitation	14
6. Remediation Roadmap	16
7. Conclusion.....	16
Appendix A: Evidence and Screenshots	17
Appendix B: Test Limitations.....	30
Appendix C: References.....	32
Appendix D: Glossary	33

1. Executive Summary

The assessor carried out a web application security assessment on the Northstar Support application, a Flask-based training environment hosted at `http://192.168.XXX.XXX:5000`. Testing focused on authentication, authorization, session management, input handling, error handling, security configuration, and monitoring using Burp Suite and manual verification techniques aligned with the OWASP Web Security Testing Guide (WSTG).

The assessment identified one confirmed **high-severity** broken-access-control vulnerability. A standard authenticated user could access another user's support ticket by changing the numeric ticket identifier in the `/tickets/<id>` route. For example, an authenticated session belonging to Alice accessed Dana's ticket through `GET /tickets/2`, even though Ticket 2 was assigned to Dana. This represents a horizontal privilege-escalation / insecure direct object reference (IDOR) issue.

The assessment also identified incomplete session-cookie hardening. The observed session cookie included the `'HttpOnly'` attribute but did not visibly include `'Secure'` or `'SameSite'`; the lab application was served over plain HTTP. SQL injection tests against the login endpoint, reflected-XSS testing of available login inputs, and vertical privilege-escalation testing against the administrator endpoint did not result in exploitation. The available standard-user functionality did not permit a full assessment of password-storage strength, stored XSS, or meaningful CSRF exploitability.

Overall risk rating: High. The confirmed ticket IDOR permits unauthorized disclosure of other users' support-ticket content.

Key Recommendations

- Enforce server-side object-level authorization for every ticket request, edit, attachment, comment, and API operation.
- Enable HTTPS and configure `Secure`, `HttpOnly`, and `SameSite` session-cookie attributes.
- Use centralized role and ownership checks rather than relying on UI visibility or route naming.
- Add authorization regression tests with multiple user roles and distinct standard-user accounts.
- Add application-level logging for authentication failures and authorization denials without storing passwords or session tokens.

2. Scope and Rules

2.1 In-Scope Assets

<u>Asset</u>	<u>Scope</u>
Northstar Support web application	http://192.168.XXX.XXX:5000
Authentication workflow	POST /login, logout/session behavior
User profile functionality	/user/<id>
Ticket functionality	/tickets/<id>
Administrator endpoint	/admin/dana
Backend database	Local SQLite database used by the authorized lab
Application logs	Local lab/server logs, where available

2.2 Out-of-Scope Activities

- Network infrastructure beyond the target web server
- Denial-of-service testing
- Social engineering
- Physical security testing
- Testing against external systems or real user data
- Production environments
- Unauthorized account access

2.3 Rules of Engagement

- The assessor performed testing only in an isolated, assessor-controlled training lab.
- The assessor only utilized authorized test accounts and fictional lab data.
- The assessor limited testing to manual web-application security verification and controlled Burp Suite requests.
- The assessor did not carry out any destructive data modification or denial-of-service activity.
- The assessor sanitized the evidence before including it in the final portfolio report.

3. Environment and Methodology

3.1 Lab Environment

<u>Component</u>	<u>Details</u>
Tester workstation	Kali Linux
Target system	Ubuntu VM hosting Northstar Support
Application framework	Python / Flask
Web service	HTTP on port 5000
Database	SQLite (instance/supporthub.sqlite)
Interception proxy	Burp Suite Community Edition
Browser	Firefox configured through Burp proxy
Burp listener	127.0.0.1:8080
Target scope	http://192.168.XXX.XXX:5000

3.2 Tools Used

<u>Tool</u>	<u>Purpose</u>
Burp Suite Community Edition	Intercepted, inspected, and modified HTTP requests and responses. Used Proxy, HTTP History, Site Map, Repeater, Intruder, and session-management features to support authentication, authorization, session, and input-validation testing.
Mozilla Firefox	Accessed the Northstar application and generated browser traffic through the Burp proxy.
Visual Studio Code	Used as the integrated development environment (IDE) for reviewing and maintaining the Flask application source code, Python scripts, HTML templates, configuration files, and SQLite database-related project files.
Visual Studio Code CLI	Used to run Linux shell commands, activate the Python virtual environment, start and stop the Flask application, execute supporting Python scripts, inspect application output, and run SQLite commands.
SQLite CLI (sqlite3)	Used for authorized local database inspection and verification of user and ticket ownership records supporting the IDOR test.

Python / Flask	Application technology assessed in the lab environment; Python scripts and Flask application components were reviewed and executed as part of the assessor-controlled lab workflow.
Linux command-line utilities	Used for authorized file navigation, process review, and log inspection in the local lab environment.

3.3 Methodology

Testing followed an OWASP-aligned workflow:

1. Reconnaissance and mapping

- Configured Burp Proxy and target scope.
- Browsed the application and reviewed the Burp Site Map and HTTP history.
- Identified authentication, ticket, profile, dashboard, logout, and admin routes.

2. Authentication testing

- Verified valid-login behavior.
- Tested invalid usernames and invalid passwords.
- Performed limited password-list testing with Burp Intruder.
- Performed controlled SQL-injection probes in the login parameters.

3. Authorization testing

- Tested horizontal authorization by modifying ticket identifiers.
- Tested profile-object access where available.
- Tested vertical authorization by requesting the admin endpoint as a standard user.

4. Session-management testing

- Inspected Set-Cookie attributes.
- Tested arbitrary invalid-cookie rejection.
- Reviewed logout behavior.
- Configured Burp session handling to maintain the authorized test session.

5. Input validation and error handling

- Tested login parameters for SQL injection and basic reflected-XSS markers.
- Reviewed invalid-route responses for verbose errors and stack traces.
- Inspected headers for security-hardening controls.

6. Logging and monitoring

- Generated controlled failed-login and unauthorized-admin-access events.
- Reviewed accessible server/application logs, where available.

4. Findings Summary

<u>ID</u>	<u>Finding</u>	<u>Severity</u>	<u>OWASP Category</u>	<u>Status</u>
F-01	Insecure Direct Object Reference (IDOR) in Ticket Detail Endpoint	High	A01:2025—Broken Access Control	Confirmed
F-02	Incomplete Session-Cookie Security Attributes and HTTP Transport	Medium	A07:2025—Authentication Failures	Confirmed
T-01	SQL Injection Testing of Login Endpoint	Informational	A05:2025—Injection	No exploitation observed
T-02	Vertical Privilege Escalation at Admin Endpoint	Informational	A01:2025—Broken Access Control	No exploitation observed
T-03	Reflected XSS Testing of Login Endpoint	Informational	A05:2025—Injection	No exploitation observed
T-04	CSRF Testing Limitations	Informational	Session Management	Not fully testable
T-05	Password Storage Review	Informational	A07:2025—Authentication Failures	Not assessed / insufficient evidence

5. Detailed Findings

F-01: Insecure Direct Object Reference (IDOR) in Ticket Detail Endpoint

Severity: High

Affected component: GET /tickets/<id>

OWASP Top 10 mapping: A01:2025—Broken Access Control

WSTG mapping: Authorization / IDOR testing

Description

The assessor identified a horizontal privilege-escalation vulnerability in the Northstar ticket-detail endpoint. While authenticated as Alice, a standard user, the assessor first accessed Alice's own ticket at:

```
GET /tickets/1
```

The assessor retained the same session, user identity, and role and changed only the numeric object identifier to:

```
GET /tickets/2
```

The application returned Dana's ticket, including ticket title, description, priority, status, owner identifier, assigned identifier, and timestamps. Database verification confirmed that Ticket 1 belonged to Alice (user ID 1) and Ticket 2 belonged to Dana (user ID 2). This demonstrates that the application did not validate ticket ownership before returning the resource.

Proof of Concept

1. The assessor authenticated as Alice using a valid standard-user account.
2. The assessor accessed `http://192.168.XXX.XXX:5000/tickets/1` and confirmed that Ticket 1 belonged to Alice.
3. The authenticated session remained the same.
4. The assessor changed the ticket identifier to request `http://192.168.XXX.XXX:5000/tickets/2`.
5. The application returned Dana's ticket with HTTP 200 OK rather than returning 403 Forbidden or 404 Not Found.
6. The assessor reviewed the local authorized SQLite database to confirm that Ticket 1 belonged to Alice and Ticket 2 belonged to Dana.

Impact

Any authenticated user may be able to enumerate and access ticket records belonging to other users. In a real support environment, this could expose confidential customer information, support requests, internal troubleshooting information, account-related details, and potentially security-sensitive content.

Remediation

Implement a server-side authorization check before returning a ticket:

```
ticket = get_ticket_by_id(ticket_id)

if ticket is None:
    abort(404)

if ticket["owner_user_id"] != session.get("user_id"):
    abort(403)
```

- Apply authorization checks to ticket views, edits, comments, file attachments, downloads, APIs, and deletion actions.
- Enforce authorization on the server for every request.
- Do not rely on hidden links, unguessable identifiers, or client-side checks.
- Create automated regression tests using at least two normal users and one administrator.
- Log denied cross-user access attempts without exposing sensitive ticket data.

Evidence (see Appendix A.4)

- Figure 11a: Alice accessed her own ticket at `/tickets/1.`
- Figure 11b. Alice's ticket list (baseline view).
- Figure 12. Alice accessed Dana's ticket after the ticket identifier was changed to `/tickets/2.`
- Figure 13. Burp Repeater evidence of cross-user ticket disclosure.
- Figure 14. SQLite user and ticket ownership mapping.

F-02: Incomplete Session-Cookie Security Attributes and HTTP Transport

Severity: Medium

Affected component: Session cookie issued after POST /login

OWASP Top 10 mapping: A07:2025—Authentication Failures

WSTG mapping: Session-management testing

Description

The captured response to a successful login set a session cookie containing the `HttpOnly` attribute and `Path=/. The observed Set-Cookie header did not visibly include Secure or SameSite. The assessor accessed the lab application through unencrypted HTTP.`

`HttpOnly` is a positive control because it limits direct access to the cookie from client-side JavaScript. However, the absence of `Secure` allows the browser to transmit the cookie over HTTP, and the lack of `SameSite` removes an important browser-level defense-in-depth control against cross-site request risks.

The login response redirected the user to `/dashboard` using `302 Found` and issued a session cookie with `HttpOnly; Path=/. Secure and SameSite were not visible in the captured response header.`

Impact

Deploying the same configuration on an untrusted or shared network would allow an attacker observing HTTP traffic to potentially capture an authenticated session cookie. Missing `SameSite` also reduces browser-based defense in depth against CSRF if state-changing features are introduced.

Remediation

Enable HTTPS in deployed environments and configure secure session settings:

```
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_SAMESITE = "Lax"
```

Use `SameSite="Strict"` where application workflows permit it. Also:

- Configure an idle timeout and an absolute session lifetime.
- Regenerate session identifiers after authentication and privilege changes.
- Do not place session tokens in URLs.
- Use server-side session invalidation or an appropriate token revocation strategy on logout.

Evidence (see Appendix A.3)

- Figure 9. Set-Cookie header showing session cookie name and attributes.
- Figures 10a–10c. Session binding, tamper resistance, and logout behavior.

T-01: SQL Injection Testing of Login Endpoint — No Exploitation Observed

Severity: Informational

Affected component: POST /login

OWASP Top 10 mapping: A05:2025—Injection

WSTG mapping: SQL injection testing

Description

The assessor tested the login email and password parameters through Burp Repeater using controlled SQL syntax and Boolean-style probes, including:

```
'  
' OR '1'='1  
' AND '1'='2
```

The application consistently returned the generic response:

```
Invalid email or password.
```

No successful authentication occurred, no redirect to /dashboard was observed for invalid test values, no authenticated session was established, and no database error messages were disclosed.

Conclusion

The assessment did not demonstrate any exploitable SQL injection or authentication bypass on the tested login endpoint.

Recommendation

- Continue using parameterized queries for all database interactions.
- Avoid dynamic SQL construction through string concatenation.
- Keep generic authentication errors.
- Add monitoring, rate limiting, and account-lockout controls appropriate to the deployment environment.

T-02: Vertical Privilege Escalation — No Exploitation Observed

Severity: Informational

Affected component: GET /admin/dana

OWASP Top 10 mapping: A01:2025—Broken Access Control

WSTG mapping: Privilege-escalation testing

Description

The assessor requested the administrator endpoint while using Alice’s authenticated standard-user session:

```
GET /admin/dana HTTP/1.1
Host: 192.168.XXX.XXX:5000
Cookie: session=abcdef123456
```

The server returned:

```
HTTP/1.1 403 Forbidden
```

No administrative content or privileged functions were disclosed.

Conclusion

When Alice’s authenticated standard-user session requested GET /admin/dana, the server returned HTTP 403 Forbidden and did not disclose administrative content or privileged functionality.

Recommendation

Continue enforcing centralized server-side role checks on all administrator routes, backend APIs, and privileged actions. Add automated tests confirming that standard users receive 403 Forbidden when requesting administrator-only functionality.

T-03: Reflected XSS Testing — No Exploitation Observed

Severity: Informational

Affected component: POST /login

OWASP Top 10 mapping: A05:2025—Injection

WSTG mapping: Reflected-XSS testing

Description

The assessor used an HTML marker to test the login email parameter in Burp Repeater, such as:

```
<xss-check>
```

The assessor submitted the marker in URL-encoded form:

```
email=%3Cxss-check%3E
```

The application returned a generic failed-authentication message and did not reflect the submitted marker in the response.

Conclusion

The tested login endpoint did not demonstrate any reflected XSS.

Limitation

The assessment did not include stored-XSS testing because the available standard-user interface did not expose a persistent user-content workflow, such as ticket creation, ticket editing, comments, or profile editing.

Recommendation

Maintain context-appropriate output encoding for all user-controlled values. Ensure Flask/Jinja automatic escaping remains enabled and only mark user input as safe after rigorous sanitization for the specific output context.

T-04: CSRF Testing Limitations

Severity: Informational

Affected component: Standard-user functionality

OWASP mapping: Session-management / CSRF testing

Description

The assessor reviewed CSRF protections during the session-management assessment. A meaningful CSRF proof of concept requires an authenticated, state-changing endpoint, such as ticket creation, profile modification, password change, or record deletion.

The available standard-user Northstar interface exposed login and read-only ticket/profile functions but did not expose a suitable authenticated state-changing workflow.

Conclusion

The assessment could not establish CSRF exploitability because the assessed functionality lacked a suitable authenticated, state-changing standard-user endpoint.

Recommendation

When the application adds state-changing functionality, protect all such requests by using server-side CSRF tokens and validation. Use SameSite cookie attributes as an additional defense layer rather than relying on them as the sole control.

T-05: Password-Storage Review Limitation

Severity: Informational

Affected component: User credential storage

OWASP mapping: A07:2025—Authentication Failures

Description

Password storage could not be fully assessed because the review did not obtain sufficient evidence about the application's password-handling implementation. The assessment did not confirm the password-hashing algorithm, whether each password uses a unique random salt, the configured work factor, or how password verification occurs.

Applications should not store passwords in readable form or use reversible encryption. Instead, they should use password hashing: a one-way process that converts a password into a stored value. When a user signs in, the application processes the submitted password in the same way and compares the result with the stored value. The original password should not need to be recovered.

Older methods such as MD5 and SHA-1 are not appropriate for password storage. They are designed to execute quickly, which enables an attacker who obtains a password database to test very large numbers of guessed passwords rapidly. Modern password-hashing algorithms—such as Argon2id, bcrypt, scrypt, and PBKDF2—are intentionally slower and can be configured to require more computing resources. This makes large-scale password guessing more costly if stored password data is exposed.

A salt is a unique random value added to each password before hashing. Salts help prevent identical passwords from producing identical stored values and reduce the usefulness of precomputed password-cracking tables. A work factor controls how computationally expensive password hashing is; it should be selected to make attacker guesses costly while keeping normal sign-in performance acceptable.

Conclusion

The assessment did not determine whether Northstar stores passwords securely or insecurely. No finding is made that Northstar uses plaintext passwords, MD5, SHA-1, or any other weak storage method.

Password-storage strength remains not assessed because of insufficient evidence. An authorized source-code review or database-format review is required to determine whether the implementation uses a modern, salted, adaptive password-hashing method with appropriate configuration.

Recommendation

Perform an authorized review of the authentication source code, password-verification logic, database schema, and any password-reset functionality.

Developers should never store passwords in plaintext or with reversible encryption. Use a modern password-hashing algorithm with a unique random salt per password and a suitably configured work factor:

- Prefer Argon2id for new implementations.
- Use bcrypt, scrypt, or PBKDF2 when they are already supported and securely configured.
- Do not use MD5 or SHA-1 as the primary password-storage mechanism.
- Protect password-reset tokens as carefully as passwords: make them random, short-lived, single-use, and securely transmitted.
- Reassess password-hashing parameters periodically as hardware performance and organizational risk requirements change.

6. Remediation Roadmap

<u>Priority</u>	<u>Timeframe</u>	<u>Action</u>
Immediate	0–2 weeks	Fix the ticket IDOR through server-side object ownership and authorization checks
Immediate	0–2 weeks	Add regression tests for cross-user ticket viewing, editing, comments, and attachments
Short term	2–4 weeks	Enable HTTPS and set Secure, HttpOnly, and SameSite cookie attributes
Short term	2–4 weeks	Review all routes for centralized role-based and ownership-based authorization
Medium term	1–3 months	Add CSRF protection to all state-changing workflows
Medium term	1–3 months	Improve application-level monitoring for failed logins and authorization denials
Medium term	1–3 months	Perform a source/database review of password storage and password-reset functionality
Ongoing	Continuous	Repeat web-application security testing after remediation and before deployment

7. Conclusion

The Northstar Support application demonstrated a significant authorization weakness in ticket access controls. The confirmed ticket IDOR permits one authenticated user to access another user's ticket record by modifying an identifier in the request URL. The development team should remediate this issue before the application is considered suitable for a production-like environment.

The admin endpoint correctly denied a standard-user session with 403 Forbidden, and the tested login endpoint did not demonstrate SQL injection or reflected XSS. However, consistent authorization enforcement is required across all object-specific routes, as demonstrated by the difference between the protected admin endpoint and the vulnerable ticket endpoint.

The application should prioritize object-level authorization, HTTPS deployment, session-cookie hardening, security-event logging, and follow-up testing after remediation.

Appendix A: Evidence and Screenshots

This appendix lists the key evidence collected during the assessment and maps each finding or test case to its supporting screenshots. All images were captured from Burp Suite or the browser during testing in the lab environment.

A.1 Reconnaissance and Scope

Proxy configuration and target scope

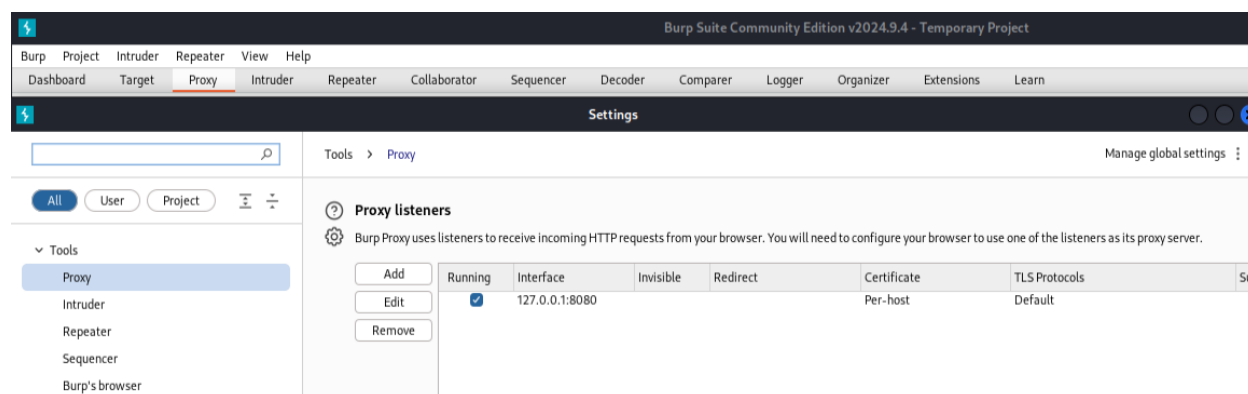


Figure 1. Burp proxy listener configuration

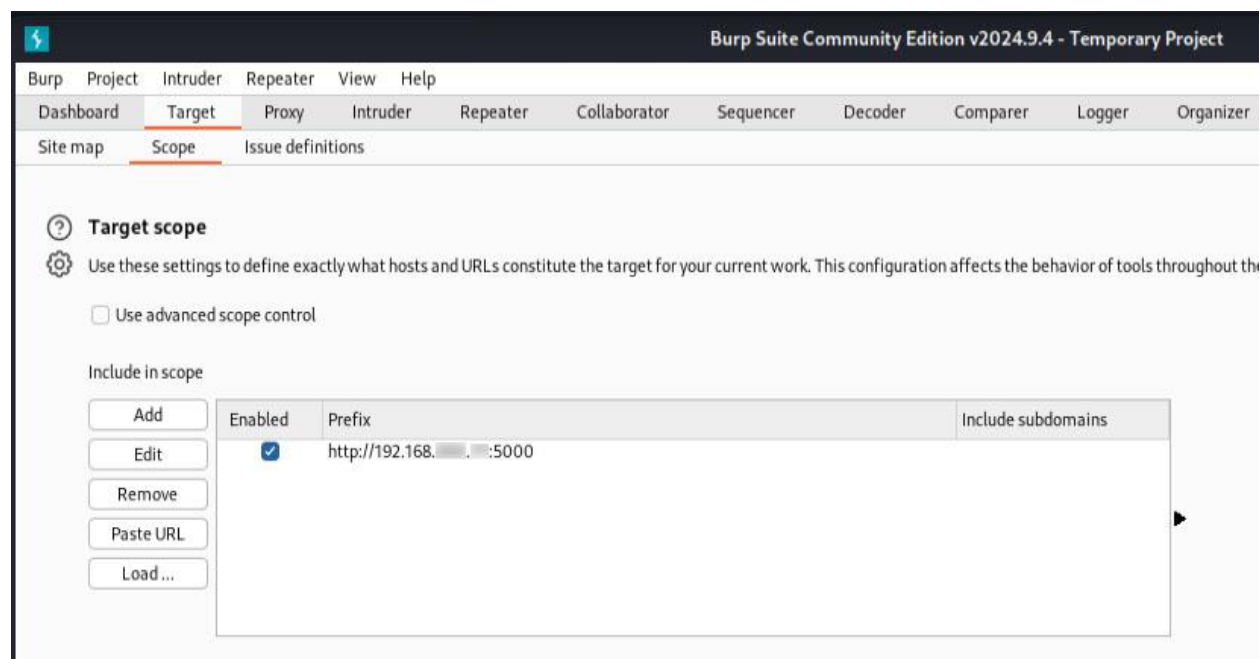


Figure 2. Defined target scope in Burp to constrain testing to authorized hosts

Application mapping

The screenshot displays the Burp Suite Community Edition v2024.9.4 interface. The top menu bar includes options like Dashboard, Project, Intruder, Repeater, View, Help, and various tool tabs. The 'Site map' tab is active, showing a tree view of the application structure on the left. The main pane displays a table of discovered paths, resources, and application structure.

Host	Method	URL	Params	Status code	Length	MIME type	Title	Notes	Time requested
http://192.168.1.100:5000	POST	/login		200	1794	HTML	Sign in — Northstar Support		17:44:06 30 Aug 2026
http://192.168.1.100:5000	POST	/login		302	513	HTML	Redirecting...		17:44:34 30 Aug 2026
http://192.168.1.100:5000	GET	/logout		302	483	HTML	Redirecting...		17:44:48 30 Aug 2026
http://192.168.1.100:5000	GET	/		200	1824	HTML	Northstar Support		17:44:56 30 Aug 2026
http://192.168.1.100:5000	GET	/about		200	1276	HTML	About — Northstar Support		17:45:04 30 Aug 2026
http://192.168.1.100:5000	GET	/user/1		200	1304	HTML	User Profile — Northstar Support		17:45:14 30 Aug 2026
http://192.168.1.100:5000	GET	/user/2		200	1302	HTML	User Profile — Northstar Support		17:45:20 30 Aug 2026
http://192.168.1.100:5000	GET	/login		200	1733	HTML	Sign in — Northstar Support		17:46:02 30 Aug 2026
http://192.168.1.100:5000	POST	/login		200	1794	HTML	Sign in — Northstar Support		17:46:29 30 Aug 2026
http://192.168.1.100:5000	POST	/login		302	513	HTML	Redirecting...		17:47:03 30 Aug 2026
http://192.168.1.100:5000	GET	/dashboard		200	1801	HTML	Dashboard — Northstar Support		17:47:05 30 Aug 2026
http://192.168.1.100:5000	GET	/static/css/styles.css		304	314				17:47:05 30 Aug 2026
http://192.168.1.100:5000	GET	/static/js/app.js		304	309				17:47:05 30 Aug 2026

The bottom section shows the 'Request' and 'Response' details for the selected path. The 'Request' tab is active, displaying the raw HTTP request. The 'Response' tab is also visible, showing the raw HTTP response.

Request:

```
1 POST /login HTTP/1.1
2 Host: 192.168.1.100:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 41
9 Origin: http://192.168.1.100:5000
10 Connection: keep-alive
11 Referer: http://192.168.1.100:5000/login
12 Cookie: session=
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 email=test%40test.com&password=testing123
```

Response:

```
29
30 <section class="form-card">
31   <p class="eyebrow">
32     Employee portal
33   </p>
34   <h1>
35     Sign in
36   </h1>
37
38   <p class="error">
39     Invalid email or password.
40   </p>
41
42   <form method="post" action="/login">
43     <label for="email">
44       Email address
45     </label>
46     <input
47       id="email"
48       name="email"
49       type="email"
50       autocomplete="username"
51     >
52   </form>
```

Figure 3. Burp site map showing discovered paths, resources, and application structure

A.2 Authentication Testing

Valid login flow

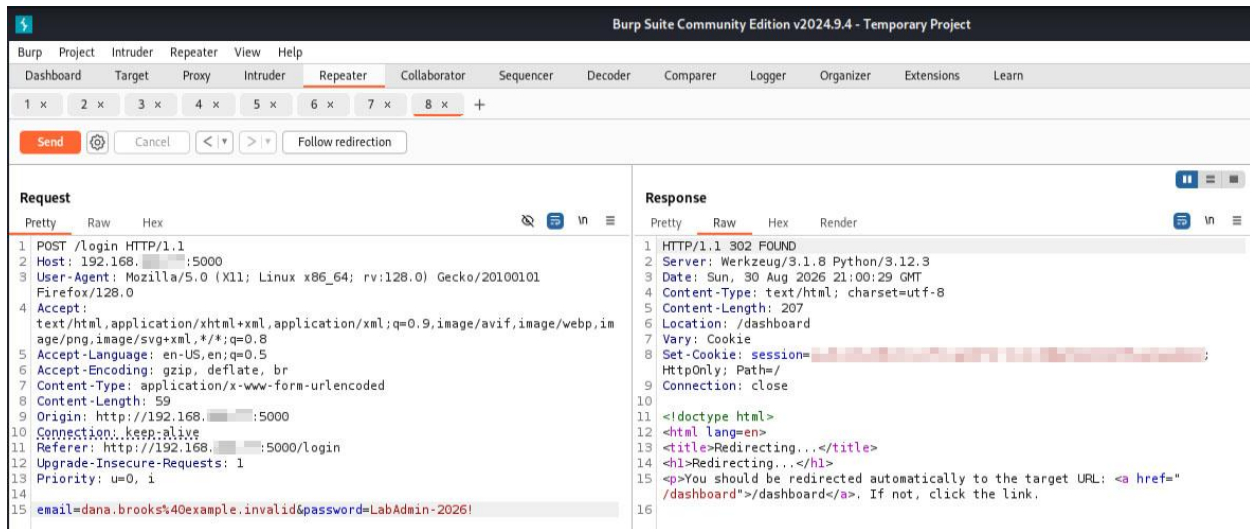


Figure 4. Successful login request and response using valid credentials

Invalid credentials handling

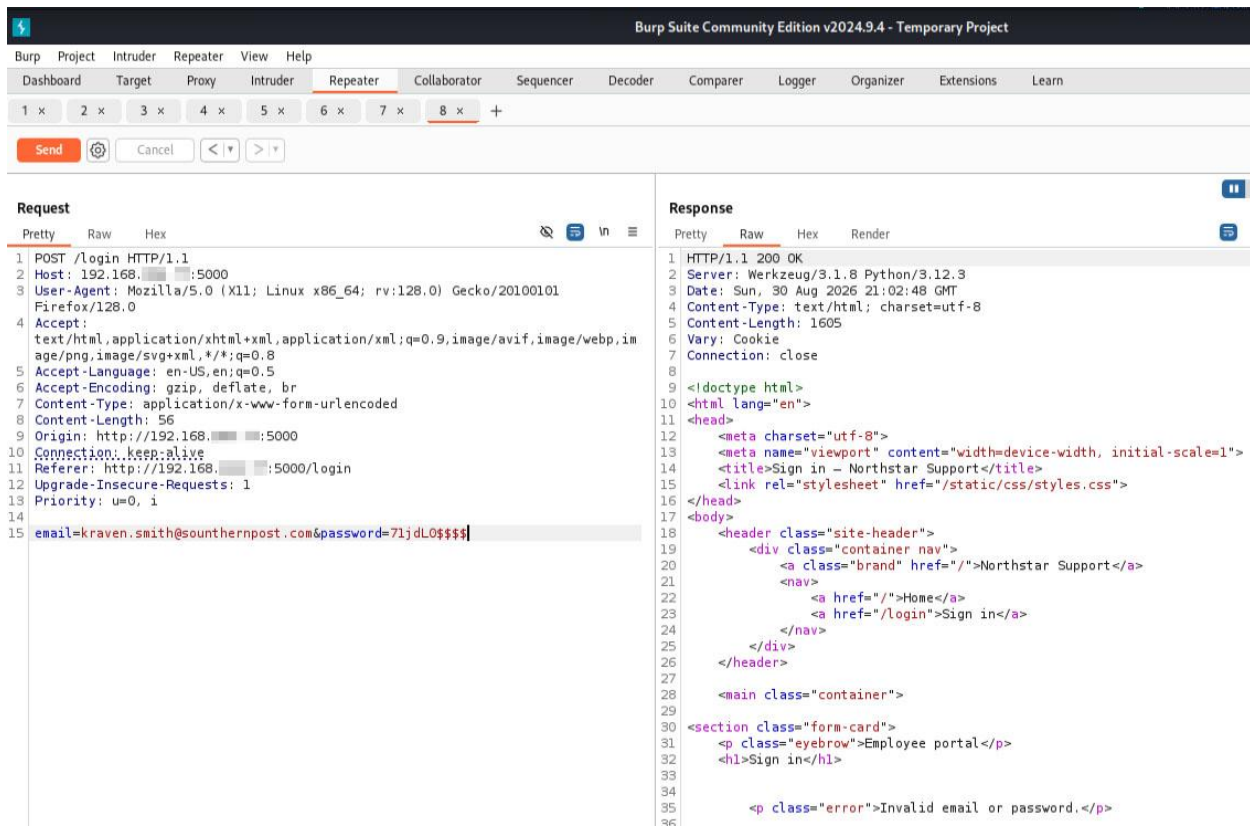


Figure 5. Login attempt with a non-existent email address


 Burp Suite Community Edition v2024.9.4 - Temporary Project

Burp Project Intruder Repeater View Help
 Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 x 2 x 3 x 4 x 5 x 6 x 7 x 8 x +

Send Cancel < >

Request

Pretty Raw Hex

```

1 POST /login HTTP/1.1
2 Host: 192.168.1.1:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
  Firefox/128.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 54
9 Origin: http://192.168.1.1:5000
10 Connection: keep-alive
11 Referer: http://192.168.1.1:5000/login
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14
15 email=dana.brooks%40example.invalid&password=987654321
          
```

Response

Pretty Raw Hex Render

```

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.12.3
3 Date: Sun, 30 Aug 2026 20:58:07 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 1605
6 Vary: Cookie
7 Connection: close
8
9 <!doctype html>
10 <html lang="en">
11 <head>
12   <meta charset="utf-8">
13   <meta name="viewport" content="width=device-width, initial-scale=1">
14   <title>Sign in - Northstar Support</title>
15   <link rel="stylesheet" href="/static/css/styles.css">
16 </head>
17 <body>
18   <header class="site-header">
19     <div class="container nav">
20       <a class="brand" href="/">Northstar Support</a>
21       <nav>
22         <a href="/">Home</a>
23         <a href="/login">Sign in</a>
24       </nav>
25     </div>
26   </header>
27
28   <main class="container">
29
30   <section class="form-card">
31     <p class="eyebrow">Employee portal</p>
32     <h1>Sign in</h1>
33
34
35     <p class="error">Invalid email or password.</p>
36
37
38     <form method="post" action="/login">
39       <label for="email">Email address</label>
40       <input
41         id="email"
42         name="email"
43         type="email"
44         autocomplete="username"
          
```

Figure 6. Login attempt with an incorrect password

Password brute-force feasibility (limited)

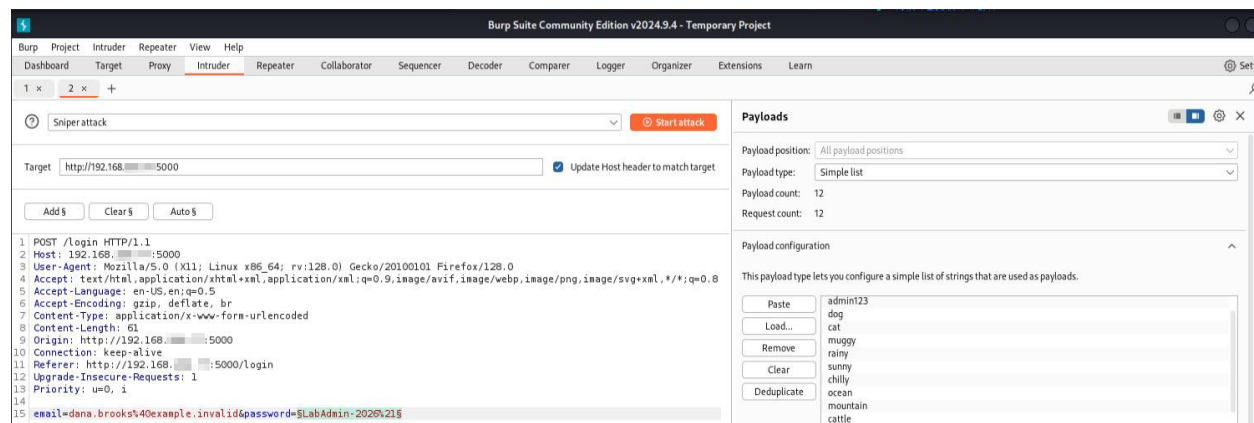


Figure 7. Intruder payload configuration targeting the password field

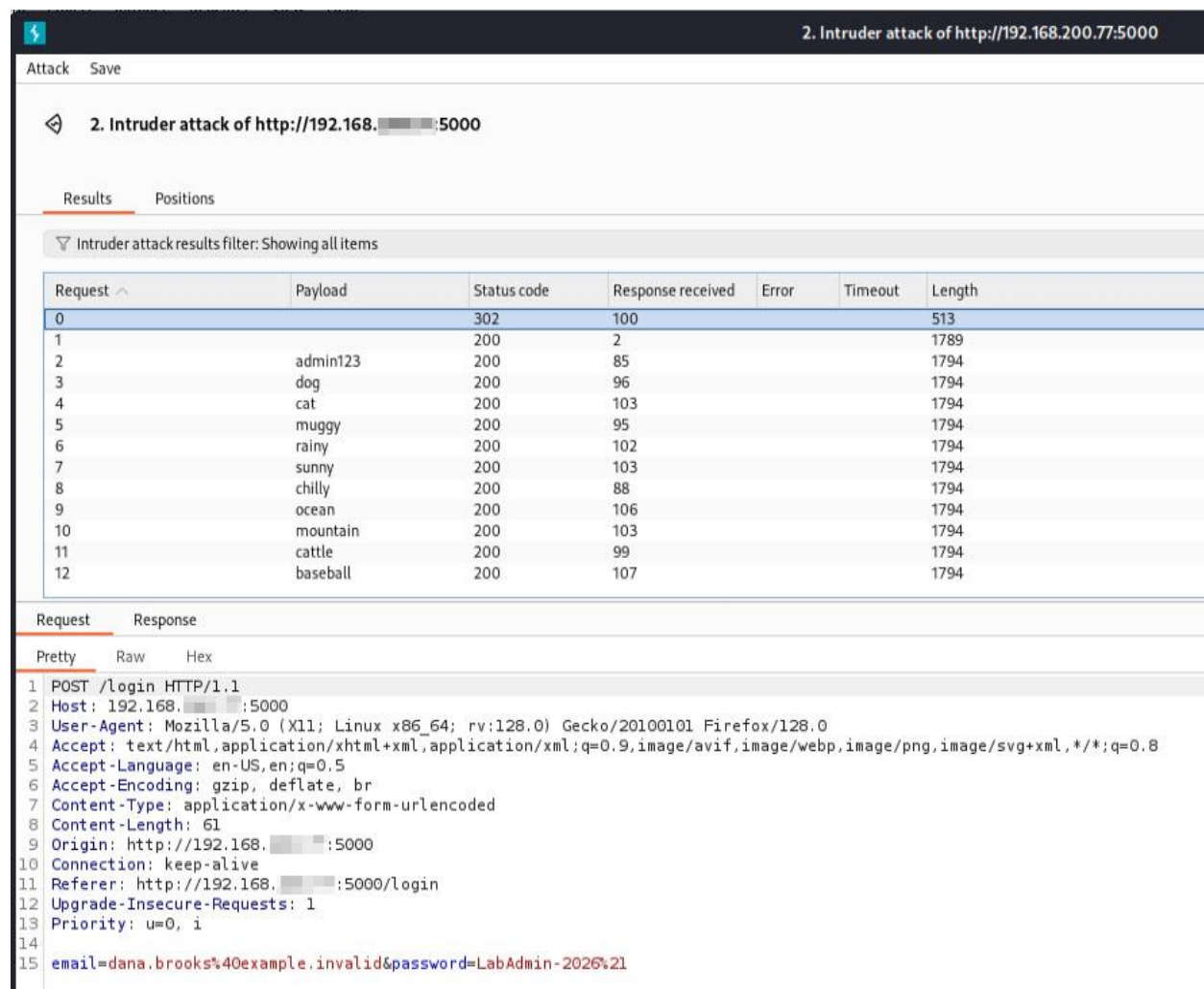


Figure 8. Intruder results table for password brute-force test

A.3 Session Management Testing

Session cookie inspection

The screenshot displays the Burp Suite interface with the 'HTTP history' tab selected. A list of intercepted requests is shown, with the third request (ID 399) highlighted. This request is a POST to the '/login' endpoint. The corresponding response is shown in the 'Response' pane, which is a 302 Found status with a 'Location' of '/dashboard'. The 'Set-Cookie' header is visible in the response, indicating a successful login and session establishment.

#	Host	Method	URL	Params	Edited	Status	Content Length	MIME type	Extension	Title	Notes	TLS IP	Cookies
402	http://192.168.1.5000	GET	/favicon.ico			404	402	HTML	ico	404 Not Found		192.168.1.5000	
401	http://192.168.1.5000	GET	/static/js/app.js			304	309	script	js			192.168.1.5000	
399	http://192.168.1.5000	GET	/dashboard			200	1801	HTML		Dashboard — Northstar Support		192.168.1.5000	
398	http://192.168.1.5000	POST	/login		✓	302	513	HTML		Redirecting...		192.168.1.5000	session=eyJlc2

Request

```
1 POST /login HTTP/1.1
2 Host: 192.168.1.5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 61
9 Origin: http://192.168.1.5000
10 Connection: keep-alive
11 Referer: http://192.168.1.5000/login
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14 email=dana.brooks%40example.invalid&
15 password=LabAdmin-2026%21
```

Response

```
1 HTTP/1.1 302 FOUND
2 Server: Werkzeug/3.1.8 Python/3.12.3
3 Date: Sun, 30 Aug 2026 20:47:01 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 207
6 Location: /dashboard
7 Vary: Cookie
8 Set-Cookie: session=eyJlc2
9 NAME: HttpOnly; Path=/
10 Connection: close
11 <!doctype html>
12 <html lang=en>
13 <title>Redirecting...</title>
14 <h1>Redirecting...</h1>
15 <p>You should be redirected automatically
16 to the target URL: <a href="/dashboard">
17 /dashboard</a>. If not, click the link.
```

Inspector

Response header

Name	Value
Set-Cookie	session=eyJlc2

Figure 9. Set-Cookie header showing session cookie name and attributes

Session binding and tamper resistance

The screenshot displays the Burp Suite Community Edition v2024.9.4 interface. The top menu bar includes Burp, Project, Intruder, Repeater, View, and Help. Below it, a toolbar shows various tools like Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Learn. The main workspace is divided into two panels: Request and Response.

Request Panel: Shows a GET /login HTTP/1.1 request. The headers include Host: 192.168.1.5000, User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0, Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8, Accept-Language: en-US,en;q=0.5, Accept-Encoding: gzip, deflate, br, Connection: keep-alive, Upgrade-Insecure-Requests: 1, Priority: u=0, i, and a custom Cookie: session=FIXED123456789SS.

Response Panel: Shows an HTML response from the application. The response includes a header with charset=utf-8, a meta viewport tag, a title 'Sign in - Northstar Support', and a link to styles.css. The body contains a site header with a navigation bar, a main container, and a form for logging in. The form has fields for email and password, both required.

Figure 10a. Request with a custom session cookie and the application's response

Figure 10b. Rejection of an invalid/tampered session cookie

Logout and session invalidation

Figure 10c. Post-logout behavior and verification that the previous session token is no longer accepted

A.4 Authorization and Access Control Testing

Horizontal privilege escalation (IDOR) on tickets

The screenshot displays the Burp Suite interface. At the top, the title bar reads "Burp Suite Community Edition v2024.9.4 - Temporary Project". The main menu includes "Burp", "Project", "Intruder", "Repeater", "View", and "Help". Below the menu is a toolbar with buttons for "Dashboard", "Target", "Proxy", "Intruder", "Repeater", "Collaborator", "Sequencer", "Decoder", "Comparer", "Logger", "Organizer", "Extensions", and "Learn". The "Proxy" tab is active, showing "HTTP history" with a filter for "Hiding CSS, image and general binary content". A table lists recent HTTP history items:

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	T
369	http://192.168.1.100:5000	GET	/static/js/app.js			304	309	script	js			
367	http://192.168.1.100:5000	GET	/tickets/1			200	1469	HTML		Ticket 1 — Northstar Support		

The "Request" pane shows the details of the selected request (ID 367):

```
1 GET /tickets/1 HTTP/1.1
2 Host: 192.168.1.100:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Cookie: session=eyJlc2... MhsgA
9 Upgrade-Insecure-Requests: 1
10 Priority: u=0, i
11
12
```

The "Response" pane shows the details of the selected response (ID 367):

```
33 Ticket #1: Alice ticket 1
34 </h1>
35 <p>
36 This is Alice's first ticket.
37 </p>
38 <p>
39 <strong>
40 Priority:
41 medium |
42 </strong>
43 <strong>
44 Status:
45 open |
46 </strong>
47 <strong>
48 Owner:
49 1 |
50 </strong>
51 <strong>
52 Assigned:
53 1
54 </strong>
55 </p>
56 <p>
57 <strong>
58 Created:
59 2026-09-04 20:18:04 |
60 </strong>
61 <strong>
62 Updated:
63 2026-09-04 20:18:04
64 </strong>
65 </p>
66 </section>
67
68 </main>
```

Figure 11a. Alice logged in and viewing her ticket (/tickets/1)

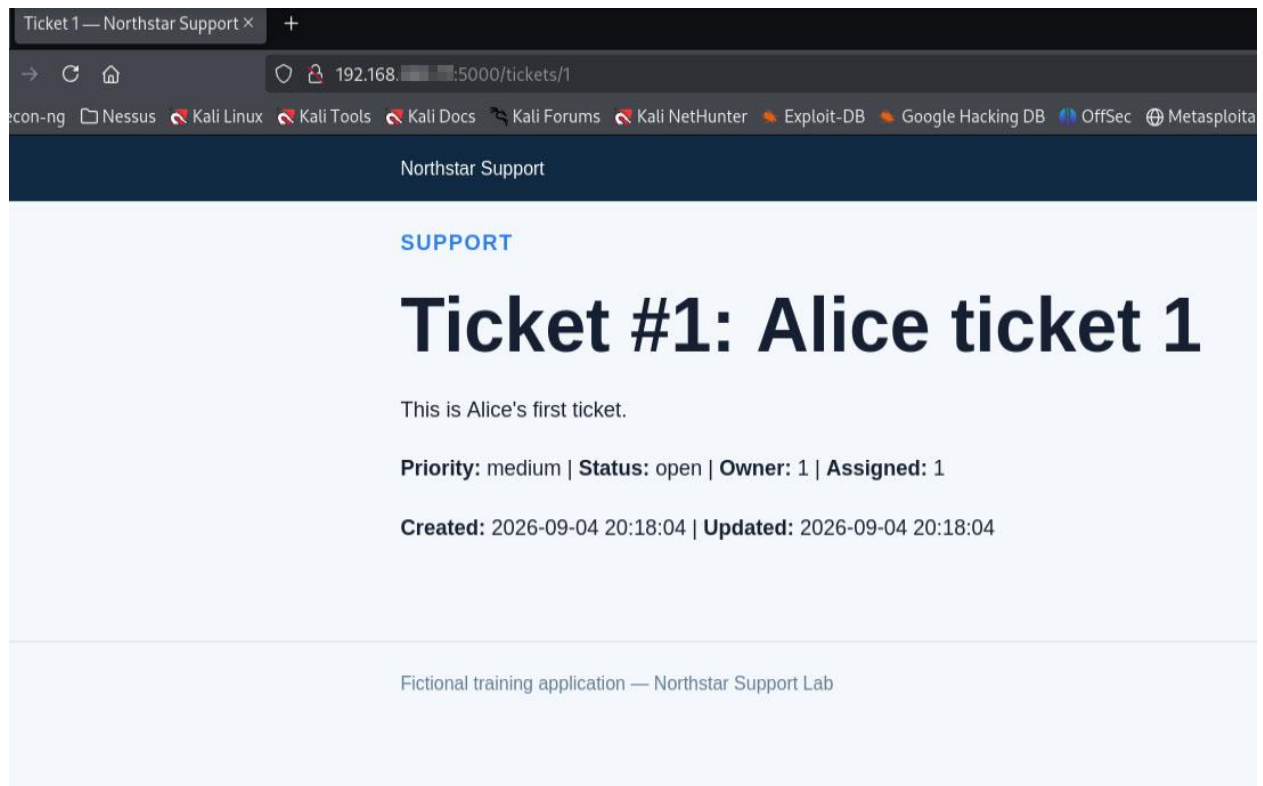


Figure 11b. Alice's ticket list (baseline view)

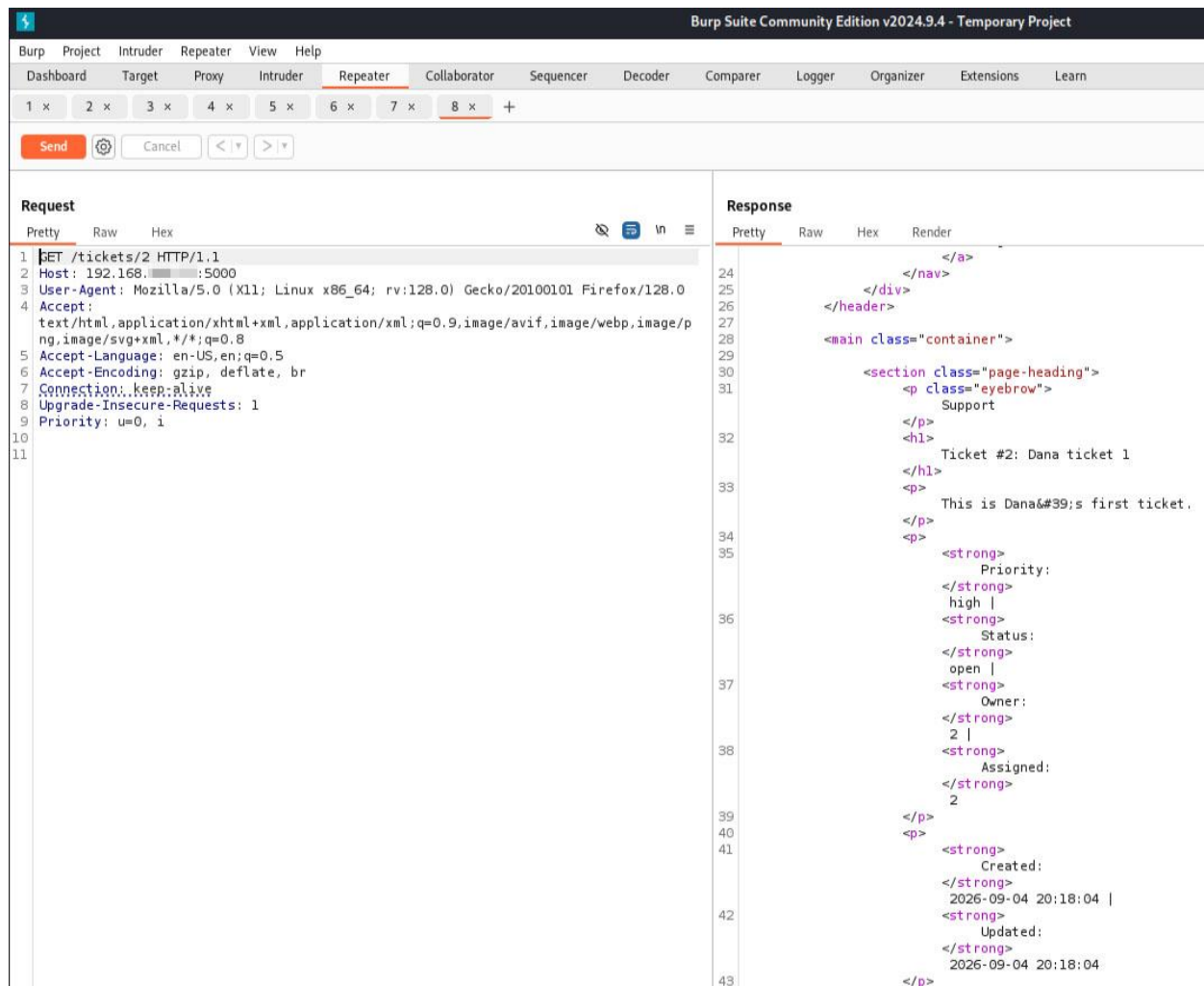


Figure 12. Access to another user's ticket by manipulating the ticket identifier (`/tickets/2`)

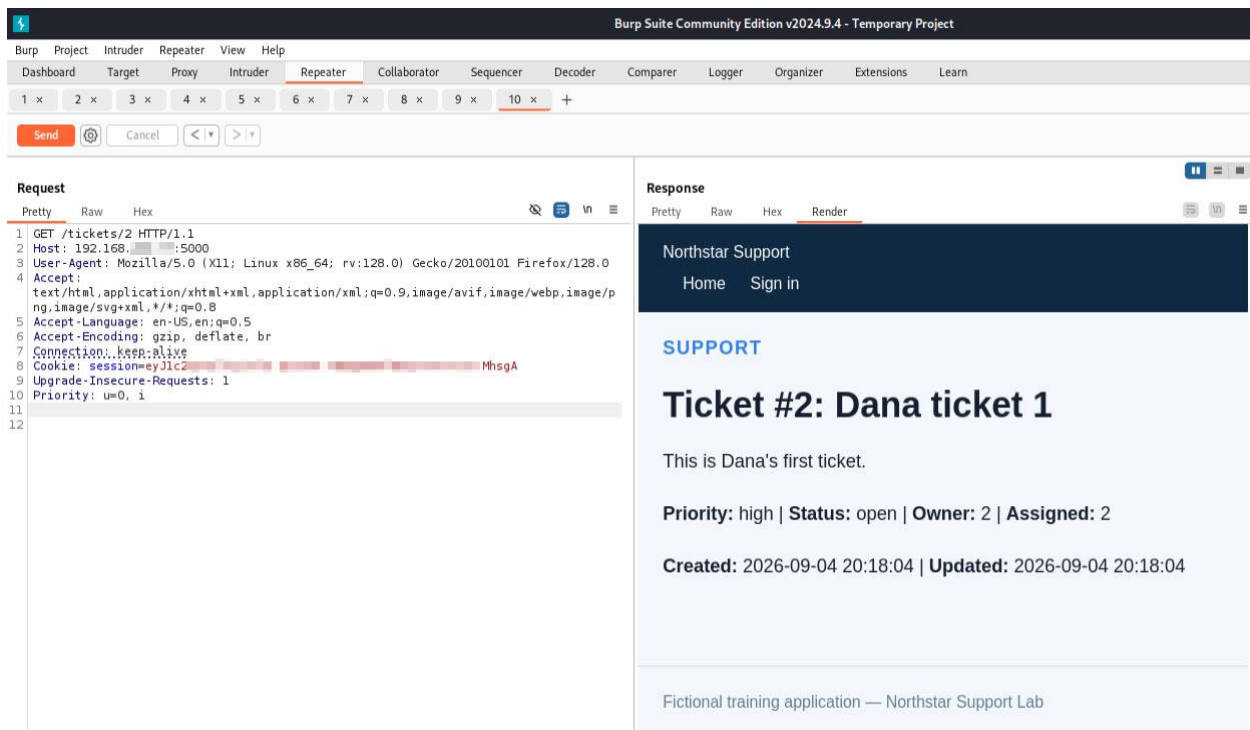


Figure 13. Repeater view confirming IDOR with rendered response

```
(.venv) -VirtualBox:~/projects/supporthub-lab$ sqlite3 instance/supporthub.sqlite "SELECT id, username, email FROM users;"
sqlite3 instance/supporthub.sqlite "SELECT id, owner_user_id, title FROM tickets;"
1|alice.morgan|alice.morgan@example.invalid
2|robert.chen|robert.chen@example.invalid
3|dana.brooks|dana.brooks@example.invalid
1|1|Alice ticket 1
2|2|Dana ticket 1
3|1|Alice ticket 2
```

Figure 14. SQLite view of the users table, confirming multiple accounts associated with tickets

A.5 Additional Tests (Negative Results)

SQL injection and XSS screening

Login-related Repeater screenshots (see Section A.2) were used to verify application responses to basic SQLi/XSS probes. No distinct additional screenshots were required for these negative results.

Vertical privilege escalation (admin route)

The admin-route test returned 403 Forbidden, indicating enforced access control. This behavior is described in the main report; no separate screenshot is included.

Appendix B: Test Limitations

The following limitations define the depth and interpretation of this assessment.

B.1 Scope and Coverage

- The assessment was time-boxed and focused on high-impact web-application controls, principally authentication, session management, and authorization.
- Not every endpoint, HTTP method, parameter, user role, workflow, or error condition was exhaustively tested.
- The assessment was conducted against the specified Northstar lab deployment; findings may not apply unchanged to another environment or production deployment.

B.2 Input Validation

- SQL injection testing consisted of basic screening on the login flow and selected inputs. Blind, time-based, second-order, and database-specific techniques were not comprehensively tested.
- XSS testing consisted of basic reflected-input screening. Stored XSS, DOM-based XSS, and context-specific encoding bypasses were not comprehensively assessed.
- CSRF defenses were not subject to dedicated, end-to-end validation.

B.3 Authentication and Sessions

- Password-storage details, including hashing algorithm, salt use, work factor, and password-reset flow security, were not independently verified.
- Intruder testing was limited to avoid account lockout, excessive authentication traffic, and disruption. A large-scale credential attack was not performed.
- Cookie behavior was evaluated from observed HTTP responses; infrastructure-level TLS termination, reverse-proxy configuration, and production security controls were not independently reviewed.

B.4 Authorization and Environment

- Access-control testing confirmed the documented ticket IDOR scenario, but it did not constitute exhaustive authorization testing across every resource type.
- Administrative functionality was tested only at the endpoint(s) available within the approved scope. The observed 403 Forbidden result is evidence for that tested route only.
- Internal network segmentation, host hardening, source-code review, dependency analysis, cloud configuration review, and post-compromise activities were outside this web assessment's scope.

B.5 Interpretation

Negative results mean that no vulnerability was identified through the techniques and coverage described above. They do not demonstrate that the application is free of vulnerabilities in those categories.

Appendix C: References

1. **OWASP Web Security Testing Guide.** Methodology and test cases for web-application security testing.
<https://owasp.org/www-project-web-security-testing-guide/>
2. **OWASP Top 10: 2025.** Includes Broken Access Control and Identification and Authentication Failures.
<https://top10.owasp.org/2025/>
3. **OWASP Authorization Cheat Sheet.** Guidance for server-side authorization and deny-by-default design.
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
4. **OWASP Insecure Direct Object Reference Prevention Cheat Sheet.** Guidance for preventing object-level authorization failures.
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
5. **OWASP Session Management Cheat Sheet.** Guidance on session identifiers, cookies, transport protection, and logout handling.
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
6. **OWASP Authentication Cheat Sheet.** Guidance on authentication design, credential handling, and defenses against automated attacks.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
7. **PortSwigger Web Security Academy: Access Control Vulnerabilities.** Background on vertical and horizontal privilege escalation and IDOR testing.
<https://portswigger.net/web-security/access-control>
8. **PortSwigger Burp Suite Documentation.** Reference for Proxy, Target, Repeater, and Intruder capabilities used during the assessment.
<https://portswigger.net/burp/documentation>

Appendix D: Glossary

This glossary defines common web-application security terms and abbreviations used throughout this report. Definitions are written for both technical and nontechnical readers.

Term	Definition
200 OK	An HTTP response status indicating that a request was successfully received and processed. In this assessment, a 200 OK response to a ticket request showed that the application returned the requested ticket data.
302 Found	An HTTP response status indicating that the browser should redirect to another location. Applications commonly return this response after a successful login to send the user to a dashboard or home page.
403 Forbidden	An HTTP response status indicating that the server understood the request but refuses to authorize the user to access the requested resource or function.
404 Not Found	An HTTP response status indicating that the requested resource could not be found. Some applications use 404 responses when a user should not be able to learn whether a protected resource exists.
Adaptive Password Hashing	A password-protection approach designed to be intentionally slow and configurable. Its computational cost can be increased over time as hardware becomes faster, making large-scale password guessing more expensive.
Administrator	A user role with elevated permissions, such as managing users, viewing administrative data, or changing application settings.
Authentication	The process of verifying a user's identity, usually with credentials such as an email address and password.
Authorization	The process of deciding whether an authenticated user is allowed to perform a particular action or access a particular resource.
bcrypt	A widely used adaptive password-hashing algorithm designed to make password guessing computationally expensive.
Broken Access Control	A security weakness in which an application does not correctly enforce what an authenticated or unauthenticated user is permitted to access or do.
Burp Suite	A web-application security testing platform used to intercept, inspect, modify, and replay HTTP requests and responses.

Burp Intruder	A Burp Suite feature used to automate controlled testing of request inputs, such as evaluating how a login endpoint responds to a limited list of password guesses.
Burp Proxy	A Burp Suite component that sits between a browser and a web application so that HTTP requests and responses can be inspected.
Burp Repeater	A Burp Suite feature that allows a tester to resend and modify an individual HTTP request, then inspect the server's response.
Client-Side Check	A restriction implemented in the browser, such as hiding a link or disabling a button. Client-side checks are not sufficient security controls because a user can alter or send requests directly.
Cookie	A small piece of data stored by a web browser and sent with later requests to the same website. Applications commonly use cookies to maintain login sessions.
Cross-Site Request Forgery (CSRF)	An attack in which a victim's browser is induced to submit an unwanted authenticated request to a website where the victim is already logged in. CSRF protections are important for requests that create, modify, or delete data.
Cross-Site Scripting (XSS)	A vulnerability that allows attacker-controlled script or markup to run in another user's browser in the context of a trusted website.
Deny by Default	An authorization design principle in which access is refused unless the application explicitly grants permission.
Encryption	A reversible process for protecting data so that it can later be decrypted with the appropriate key. Encryption alone is not an appropriate way to store user passwords because passwords should not need to be recovered.
Endpoint	A specific application route or URL that receives a request and performs a function, such as a login page, ticket-detail page, or administrator page.
Flask	A Python web framework used to build web applications and APIs.
Hash	The fixed-length output produced when data is processed through a hashing function. For password storage, the stored hash is used to verify a submitted password without storing the original password in readable form.
Horizontal Privilege Escalation	Accessing another user's resources while remaining at the same permission level. For example, one standard user viewing another standard user's support ticket.

HTTP	Hypertext Transfer Protocol, the protocol used for communication between a browser and a web server. HTTP does not encrypt information in transit.
HTTPS	HTTP protected by TLS encryption. HTTPS helps protect credentials, session cookies, and other information while it travels between a browser and a server.
HttpOnly	A cookie attribute that prevents client-side JavaScript from directly reading the cookie. It provides an important layer of defense if a cross-site scripting vulnerability exists.
IDOR (Insecure Direct Object Reference)	A broken-access-control vulnerability in which an application uses a user-supplied identifier, such as a ticket number, without verifying that the current user is authorized to access the corresponding object.
Least Privilege	The principle of granting users, services, and processes only the permissions they need to perform their intended tasks.
MD5	An older, fast hashing function that is not suitable for password storage. Its speed allows attackers to test password guesses very quickly if password-hash data is exposed.
OWASP	The Open Worldwide Application Security Project, a nonprofit organization that publishes widely used web-application security guidance, testing resources, and awareness materials.
OWASP Top 10	An OWASP awareness document that identifies major categories of web-application security risks.
Parameterized Query	A database-query technique that keeps the query's instructions separate from supplied input data. It helps prevent SQL injection vulnerabilities.
Password Hashing	A one-way process that transforms a password into a stored value. During login, the application hashes the submitted password and compares the result with the stored value; it should not need to recover the original password.
PBKDF2	A password-based key-derivation function that repeats hashing many times to increase the computational cost of password guessing.
Plaintext Password	A password stored in readable, unprotected form. Applications should never store user passwords in plaintext.
Privilege Escalation	Gaining access to data, functions, or permissions beyond those assigned to the current user.
Proof of Concept (PoC)	A controlled demonstration showing that a security issue can be reproduced under the assessment conditions.

Role-Based Access Control (RBAC)	An authorization approach that grants permissions based on a user's assigned role, such as standard user, support agent, or administrator.
Salt	A unique random value combined with a password before hashing. A salt helps ensure that identical passwords do not produce identical stored values and reduces the usefulness of pre-computed password-cracking tables.
SameSite	A cookie attribute that controls when a browser sends a cookie with cross-site requests. It provides browser-level defense in depth against some CSRF scenarios.
Secure Cookie Attribute	A cookie attribute that instructs a browser to send the cookie only over HTTPS connections.
Session	A mechanism used by an application to maintain a user's authenticated state across multiple requests.
Session Cookie	A cookie that identifies or helps maintain an authenticated browser session. If it is disclosed to an attacker, it may allow that attacker to impersonate the logged-in user.
Session Fixation	An attack in which an attacker attempts to cause a victim to use a session identifier known to the attacker. Regenerating a session identifier after login helps reduce this risk.
Session Invalidation	The process of ending a session so that its identifier can no longer be used. A secure logout should invalidate the server-side session or otherwise revoke the associated token.
SHA-1	An older hashing function with known cryptographic weaknesses. Like MD5, it is too fast to use as the sole mechanism for password storage.
scrypt	A password-hashing algorithm designed to require substantial memory as well as processing time, increasing the cost of large-scale password guessing.
Server-Side Check	A security control enforced by the application server after it receives a request. Server-side authorization checks are required because a user can bypass browser-based restrictions and send requests directly.
SQL Injection (SQLi)	A vulnerability that occurs when untrusted input changes the intended meaning of a database query. Successful SQL injection may enable unauthorized data access, modification, or authentication bypass.
Standard User	A normal application user without administrative permissions. A standard user should be able to access only the data and functions assigned to that account.

TLS	Transport Layer Security, the cryptographic protocol commonly used to secure HTTPS connections.
Token	A value used to represent a session, confirm a request, reset a password, or authorize an action. Tokens should be random, protected from disclosure, and limited in lifetime and scope.
URL	Uniform Resource Locator, commonly called a web address. In this assessment, ticket identifiers appeared in URLs such as /tickets/1.
Vertical Privilege Escalation	Accessing functions reserved for a higher-privileged role. For example, a standard user accessing an administrator-only route would be a vertical privilege-escalation issue.
Work Factor	A setting that determines how computationally expensive password hashing is. A suitable work factor makes an attacker's password guesses costly while keeping normal sign-in performance acceptable.